

Contents

Detecting self-indexing in LLM reasoning traces: experimental protocol	2
Introduction	2
Part I: Framing	3
1. Hypothesis	3
2. Why this is not game-tree search	4
3. Routine vs. research mathematics	5
4. Two axes: task structure and addressing structure	5
5. Core constructs	6
Part II: The protocol	7
6. Stage 0: precondition: using a given exploration log	7
6.1 Minimal toy proof system	8
6.2 Black-box behavioural experiment	8
6.3 Stage-0 controls	10
6.4 Dataset generation	11
6.5 Trace-format probe: do explicit handles help	11
7. Stage 1: self-generated trace and access-depth	12
7.1 Task	12
7.2 Clean/corrupt within the self-generated trace	12
7.3 The access-depth measurement (the heart of the protocol)	13
7.4 Revival variant: non-monotonic search	13

8. Stage 2: spontaneous handles: detection and ablation	14
9. Stage 3: varying the addressing structure	15
Part III: Shared methods	16
10. Activation patching	16
11. Linear probing	17
12. Attention and retrieval diagnostics	18
13. Latent vs. written state	18
14. Metrics, controls, statistics	19
Part IV: Reading the results	19
15. Pass / fail	19
16. Expected outcomes	21
17. Implementation notes	21
18. Target claim	26

Detecting self-indexing in LLM reasoning traces: experimental protocol

Introduction

This document specifies an experimental protocol for one question: when a language model runs a long search on a hard problem, does it build an internal **index** over its own reasoning trace: a structure that keeps the whole search, detours and abandoned branches included, cheaply re-accessible and, when later progress warrants, revivable?

It is organized in four parts. **Part I** fixes the framing: the hypothesis, why the object of study is unlike game-tree search,

the routine/research-mathematics demarcation it implies, and the two axes, task structure and addressing structure, that the design must keep separate. **Part II** is the protocol proper, a four-stage progression: Stage 0 checks a precondition: whether a model can use a search-state annotation *handed to it*; Stages 1–3 put the model in the driver’s seat of its own search and measure whether it indexes the trace it itself produces. **Part III** collects the methods: activation patching, probing, diagnostics, and statistics shared across stages. **Part IV** says how to read the results.

Part I: Framing

1. Hypothesis

While generating a long proof attempt, a model builds an **index over its own reasoning trace**, implicitly in its activations: a structure that keeps any earlier point of the search (a step, a detour, an abandoned branch) cheaply re-accessible as the trace grows, rather than requiring a linear re-scan.

Motivating analogy: code already ships with such an index: the call/dependency graph plus text searchability. The conjecture is that strong reasoning models *internalize the same trick* for their own thoughts, and that this is part of why coding competence came first and mathematical competence is following.

“Logarithmic access” is not a claim about wall-clock time (a forward pass is fixed-cost). It is a claim about **circuit depth**. A transformer of depth L realizes any internal computation as a circuit of depth $\leq L$. Routing or aggregating information over a trace of n positions then has a real complexity:

- **$O(1)$ depth**: direct attention to a known position.

- **$O(\log n)$ depth:** hierarchical aggregation/ search through landmark representations: a balanced-tree computation, $\sim \log n$ composed attention hops.
- **$O(n)$ depth:** sequential scan: exceeds fixed depth, impossible in one pass (this is *why* externalized chain-of-thought exists).

The hypothesis, sharply: the model organizes its trace so that earlier content becomes usable in **$O(\log n)$ depth** through multi-scale landmark/summary representations distributed across the residual streams of the trace, rather than $O(1)$ -but-imprecise (soft attention dilutes when selecting one item among many) or $O(n)$ -impossible.

2. Why this is not game-tree search

This is the distinction that separates the hypothesis from the search procedures it superficially resembles. MCTS and alpha-beta are **monotonic** and **frontier-local**: a pruned branch stays pruned, a node's value is a function of its own subtree, and the state needed to expand the frontier is local to the path under examination; backtracking is just popping a stack. Branches are independent subtrees, coupled only by the value propagated toward the root.

Proof search is neither. Branch *viability* is **non-monotonic**: a branch closed for want of a lemma becomes live once that lemma appears. And the search is **non-local**: a branch's status depends on the global pool of results obtained *anywhere* in the search, because any result is a reusable lemma. Branches are coupled through a shared, growing fact pool, not through a value.

This is why a *frontier*, the game-search notion, is too weak an object. Search state cannot be kept only at the frontier point under examination; the entire explored region, abandoned branches included, must stay cross-referenceable, so that a new fact can be matched against the closure reasons of branches closed long ago. Backtracking does not reach there; **revival** does. Revival is content-triggered, so it requires an associative, content-addressed index over the *whole history*. That is the index this protocol is built to detect, and revival (§5, §7.4) is its operational signature.

3. Routine vs. research mathematics

The monotonic/non-monotonic split is almost a demarcation of *research* mathematics from *routine* mathematics. Routine mathematics, such as applying a known method, executing a known proof schema, or computation, is largely frontier-local and monotone: the relevant state is local, dead-ends are shallow, success is recognised near the frontier. It is exactly the regime where game-tree search, automated theorem provers, and now LLMs already do well, and where competence is drillable and RL-friendly.

Research mathematics is defined by the part that is not: progress is non-monotone (a long-abandoned line revives) and non-local (the unblocking idea comes from a different branch, often a different field). The OpenAI unit-distance disproof is itself this pattern: number-theoretic machinery surfacing to settle a discrete-geometry problem, a transfer the remarks paper notes no human had made.

This sharpens the explanandum. “LLMs are increasingly strong at mathematics” is unsurprising for the routine regime and says little. The substantive claim, and what the revival variant (§7.4) actually probes, is whether a model can do the *non-monotone, non-local* part. The indexing hypothesis then doubles as a demarcation: a model that maintains a genuine content-addressed index over its search history has the structural prerequisite for research-type mathematics; a model confined to frontier-local search does not, however fluent at routine work.

4. Two axes: task structure and addressing structure

Coding is not the monotone regime: debugging is itself a paradigm of non-monotone, non-local search (the cause is typically far from the symptom, and abandoned approaches revive). Two *orthogonal* axes are in play and must not be conflated:

- **Task structure:** monotone/frontier-local vs non-monotone/non-local. The routine-vs-research axis of §3. A property of the *problem*.

- **Addressing structure:** whether the explored material comes with an explicit, efficient index (random-access handles, navigation, search) or not. A property of the *medium*.

Code’s distinguishing feature is the **addressing structure**, not the task structure: a codebase ships an obvious, efficient index: the call/dependency graph, the file tree, symbol names, text search, and jump-to-definition. So even non-monotone code work (debugging, refactoring) can lean on a *given* index. Mathematics supplies no such thing; an exploratory proof has no call graph and no search over its own history.

The internal-index hypothesis is therefore load-bearing in exactly one cell: **non-monotone task + no external addressing structure**, i.e. research mathematics. That is where, if the model copes at all, it must have built the index itself. Code is the right contrast not because it is monotone but because there the index is supplied; the experiment must vary the *addressing structure* while holding the task non-monotone (see Stage 3, §8).

5. Core constructs

Self-generated trace. From Stage 1 on, the model produces the reasoning trace itself by working a genuine multi-step search task over many tokens. All probing and patching then happens *inside that self-generated trace*. (Stage 0 is the exception: there the trace is supplied.)

Callback fact. Each task plants a fact φ established early in the search (position p) that becomes decision-relevant much later (position q), at trace distance $d = q - p$. The model must use φ at q to act correctly. This is the unit on which access is measured.

Revival trigger. Search is treated as *non-monotonic*: a closed branch is not permanently gone. Each closed branch carries an explicit *reason for closure*: a missing lemma, an undischarged side condition. A revival trigger is a fact ρ , surfaced later while exploring a *different* branch, that discharges that reason; correct behaviour is then to **revive** the previously closed branch. This

is the strongest stress test of the index: an abandoned branch must stay addressable *and* re-evaluable, so that new content elsewhere can reactivate it. Distinguish **refuted** branches (genuinely impossible, never revivable; used as corrupt-condition distractors) from **blocked** branches (closed for a dischargeable reason, revivable).

Availability depth $\ell^*(\mathbf{d}, \mathbf{n})$. The central quantity of Stages 1–3. For a callback fact φ at distance $\mathbf{d}$ in a trace of length $\mathbf{n}$, train layer-wise linear probes to decode φ from the residual stream at the decision position $\mathbf{q}$. ℓ^* is the minimal layer at which φ becomes linearly decodable above control baselines. Its scaling shape is the experiment:

- $\ell^* \approx \text{constant}$, low $\rightarrow$ flat / direct access.
- ℓ^* grows $\approx \log \mathbf{d}$ (and is robust as $\mathbf{n}$ grows) $\rightarrow$ **hierarchical index**.
- φ never decodable at $\mathbf{q}$ without the model re-deriving it in emitted tokens $\rightarrow$ no internal index; the trace is a flat list and CoT carries everything.

Handle. A token span the model itself emits that plausibly functions as an index entry: a summary line, a named lemma / case (“Case 2:”, “Lemma A”), a status note (“this branch fails because...”), an explicit callback (“returning to the idea that...”). Handles are *model-emitted*, not experimenter-inserted.

Part II: The protocol

6. Stage 0: precondition: using a given exploration log

Stage 0 tests a precondition, not the hypothesis: can a model comprehend and act on an externally supplied search-state annotation at all? The exploration log is **handed to the model in the prompt**; the model only reads a verdict and acts. If

Stage 0 fails, the model is too weak for the toy calculus and the calculus must be simplified before Stages 1–3 are interpretable. **Passing Stage 0 establishes nothing about self-indexing**; only that the model can use a given log.

6.1 Minimal toy proof system

A synthetic Horn-clause-style forward proof system, deliberately simple. A task contains a start fact S , a goal G , a set of labelled actions/ rules, an exploration log saying which branches have been tried and whether they are dead or viable, and a current obligation requiring the model to choose the next action. A branch is dead **iff the exploration log marks it dead**; this avoids a confound where the model derives deadness from the rule list and ignores the exploration state.

Example. Start node S ; first actions $R1: S \rightarrow B1$, $R2: S \rightarrow B2$; continuations $B1 \rightarrow T1$, $B2 \rightarrow T2$. The terminal verdicts are given only by the log: $T1 = \text{viable}$, $T2 = \text{dead}$ (or swapped in the corrupt condition). The model chooses between $R1$ and $R2$ at S .

6.2 Black-box behavioural experiment

Use prompts with single-letter answer choices so candidate log-probabilities are easy to measure.

You are doing proof search in a toy calculus.

A branch is dead only if its exploration record says STATUS: DEAD.

A branch is viable only if its exploration record says STATUS: VIABLE.

When choosing the next move, avoid branches already known to be DEAD.

Rules:

$R17: S \rightarrow B17$

R04: S -> B04
R21: B17 -> T17
R88: B04 -> T04

Exploration log:

<BRANCH B04>

Start action: R04

Path: S -> B04 -> T04

STATUS: DEAD

Reason: failed side condition C04

</BRANCH>

Current obligation:

We are again at S.

Choose the next action.

Available actions:

(A) apply R17

(B) apply R04

Answer with exactly one letter: A or B.

Clean/corrupt pair. Same rules and same final suffix, with swapped exploration verdicts. *Clean*: B04 marked DEAD → correct answer A. *Corrupt*: B17 marked DEAD → correct answer B. The final query suffix must be **byte-identical** after the exploration

log, so the only difference is the earlier log.

Behavioural metric. For each prompt P , the candidate logit margin

$$\Delta(P) = \log p(a^* | P) - \max_{a \neq a^*} \log p(a | P),$$

which for two choices is $\log p(a^* | P) - \log p(a_{\text{wrong}} | P)$. Positive means the model prefers the correct next action. Main signatures: $\Delta(P_{\text{clean}}) > 0$ and $\Delta(P_{\text{corrupt}}) > 0$ despite the correct answer differing. Also measure the **flip-rate**, $\Pr[\arg\max p(\cdot | P_{\text{clean}}) \neq \arg\max p(\cdot | P_{\text{corrupt}})]$; a high flip-rate means the model uses the exploration log.

6.3 Stage-0 controls

- **No-log:** remove the exploration log; the model should be near balanced after answer-choice randomisation.
- **Irrelevant-log:** mark dead a branch not among the current actions; the model should not systematically change its answer.
- **Length-matched corrupt:** same token count, same number of branch records, same status words, same suffix; only *which* branch is dead changes. Blocks length/formatting artefacts.
- **Label randomisation:** randomise rule labels, branch labels, and answer-choice order per instance. Blocks R1/A/first-listed biases.
- **Paraphrase:** express the same abstract state several ways (STATUS: DEAD; “Verdict: this branch is closed.”; “This line of search cannot lead to the goal.”; “Mark branch B04 as refuted.”). If the model encodes an abstract state, performance survives paraphrase.
- **Contradictory-rule** (diagnostic, not benchmark): make the rule list imply one verdict and the log state the opposite, to separate reliance on the trace from independent symbolic derivation.

6.4 Dataset generation

Generate instances over: first-step branches $b \in \{2, 4, 8\}$; logged branches $n \in \{1, 2, 4, 8, 16, 32, 64\}$; success-chain length $L_s \in \{2, 3, 4\}$; dead-branch length $L_d \in \{2, 3, 4\}$; answer choices $k \in \{2, 4\}$; trace format $\{\text{raw}, \text{labelled}, \text{summary}, \text{graph}\}$.

Per instance: sample a fresh symbol set $\{S, G, B_i, T_i, C_i\}$; sample b first actions $R_i: S \rightarrow B_i$; sample a short continuation per branch; choose a viable target branch i^* and at least one competing branch marked dead in the clean log; build the corrupt pair by moving the dead mark to i^* ; randomise the answer-choice mapping; keep the query suffix identical across clean/corrupt.

Minimal dataset: 1,000 paired prompts black-box; 5,000 probing; 1,000 held-out probing; 500 patching. Stronger: 10,000 probing; 2,000 held-out; 2,000 patching. **Splits:** disjoint by graph template *and* symbol vocabulary; hold out entire structural templates (branch depth, number of dead/distractor branches, record order, trace format), not merely renamed symbols, so the test measures abstraction rather than string memorisation.

6.5 Trace-format probe: do explicit handles help

Compare four formats of the *given* log, at increasing logged-branch count n : **raw** ("S went to B04, then B04 went to X04 ... This did not work."); **labelled** (Branch B04: $S \rightarrow B04 \rightarrow X04 \rightarrow T04$. Status: DEAD.); **labelled + summary** (adds Reason: and Summary: do not use R04 from S.); **explicit dependency graph** (nodes, actions, statuses, and an explicit Current frontier).

Prediction: $\text{accuracy_graph}(n) > \text{accuracy_summary}(n) > \text{accuracy_labelled}(n) > \text{accuracy_raw}(n)$, and crucially the *scaling* is gentler for handled formats: $|\text{d accuracy_graph} / \text{d log } n| < |\text{d accuracy_raw} / \text{d log } n|$. Note this tests whether *experimenter-inserted* handles help; Stage 2 (§7) tests the model's *own* handles, which is the actual self-indexing question.

7. Stage 1: self-generated trace and access-depth

From here the model drives the search. Probing and patching happen inside the trace the model itself produced.

7.1 Task

A synthetic forward-search calculus the model solves itself. Per instance: a start fact S , a goal G , a labelled rule set; genuine branching (several first moves, only some leading to G); the model is instructed to search step by step, record what it tries, backtrack from dead ends, until it reaches G . No exploration log is supplied; the model produces its own trace, detours and abandoned branches included.

A **callback** is built in: a fact φ discovered while exploring an early branch (e.g. “rule R consumes resource C ”) that, d tokens later, determines the correct move in a different branch. Vary d by padding intervening search with controlled irrelevant exploration; vary trace length n independently.

7.2 Clean/corrupt within the self-generated trace

Build clean/corrupt pairs by intervening on the model’s *own* trace:

1. Let the model generate a trace; identify the token span where φ is established.
2. **Clean** = the trace as generated. **Corrupt** = the same trace with φ ’s span minimally edited to the opposite fact, everything after held byte-identical up to the decision point q .
3. Re-run the forward pass on both; the correct action at q differs. Measure the candidate logit margin Δ and flip-rate as in §6.2.

This isolates the causal contribution of a fact buried d tokens back in the model’s own reasoning. Generate the trace once per

instance and freeze it; clean/corrupt are edits to a frozen trace so everything downstream is held byte-identical up to q .

7.3 The access-depth measurement (the heart of the protocol)

For each callback at distance d in a trace of length n :

1. **Layer-wise decodability.** Train linear probes $\varphi \leftarrow h(\ell, q)$ for every layer ℓ , on the residual stream at the decision position q . Record $\ell^*(d, n) = \text{minimal layer beating controls}$.
2. **Routing / hop count.** Use path patching across *positions* to trace how φ reaches q . Direct 1-hop routing vs. routing through a small set of intermediate positions (candidate index nodes / handles) discriminates flat access from hierarchical access.
3. **Patch-rescue localization.** Apply the ρ score (§9) over the self-generated trace: does rescue occur at the literal φ tokens (surface), at intermediate summary/handle tokens (index nodes), or at q (abstract)?

Primary deliverable: the $\ell^*(d, n)$ surface and the hop-count curve. Their *shape* is the test of the indexing hypothesis (see §1).

7.4 Revival variant: non-monotonic search

Instances in which a branch Y is closed early with reason “blocked: needs L ”, and later, while the model explores a different branch X , a sub-result discharges L . The model is given no prompt that L bears on Y ; recognising the connection and reopening Y is the task.

- **Behavioural metric: revival-rate.** Once ρ is in the trace, the probability mass the model places on the Y -resuming action.
- **Clean/corrupt.** Clean = ρ present (revive Y); corrupt = ρ absent or altered so L stays undischarged (do not revive). Correct behaviour differs; measure the flip, as in §7.2.

- **Distance scaling.** Vary the trace distance between Y 's closure and ρ . An index predicts revival survives large distance; a recency mechanism predicts revival decays with distance like any non-indexed retrieval.
- **Re-access vs. re-derivation control (essential).** The model could “revive” Y by regenerating it from scratch rather than re-accessing the abandoned branch, which would not be evidence of an index. Distinguish them: path-patch from Y 's original token span to the revival decision (genuine re-access routes through the old span; re-derivation does not), and check whether the revived continuation reuses Y 's specific labels and sub-results (re-access) or only its conclusion (re-derivation).
- **Refuted distractor.** Include genuinely refuted branches paired with a surfaced fact that looks relevant but discharges no real closure reason; a correctly-indexed model does not revive these.

Revival is the sharpest single discriminator in the protocol: a flat “dead = forget” trace cannot revive at all; a recency-only mechanism revives only textually-near branches; a genuine index revives the *right* closed branch at arbitrary distance, triggered by content.

8. Stage 2: spontaneous handles: detection and ablation

This inverts §6.5. There the experimenter inserts handles and asks if they help; here we ask whether the model **emits its own** handles and **relies on them**.

Detection. On long self-generated traces, annotate candidate handle spans: summary lines, named cases/lemmas, status notes, explicit callbacks. Use a combination of surface patterns, a learned span classifier, and the model's own structural markers. Report handle density as a function of trace length and task difficulty (prediction: density rises with n).

Ablation (the decisive Stage 2 test). For traces with callbacks at distance d : *handle-intact* = trace as generated; *handle-ablated* = the model's own handle spans replaced by length-matched, semantically inert filler (token count, formatting, position preserved). Re-run from the ablation point; measure Δ and accuracy. Predictions if handles function as an index:

- ablation degrades accuracy, and the degradation **scales with d and n** ; handles matter *more* the longer the trace / the further the callback;
- after ablation, $\ell^*(d, n)$ rises sharply or φ becomes undecodable at q ;
- later decisions route disproportionately through handle tokens (attention + path patching), and ablating *handle* tokens hurts far more than ablating length-matched *non-handle* tokens.

Control. Ablate length-matched non-handle spans as the baseline. Self-indexing predicts a large gap (handle ablation $\gg$ non-handle ablation), growing with n .

9. Stage 3: varying the addressing structure

Run the Stage 1 access-depth and revival protocol on a **non-monotone, non-local search task held fixed**, presented in two regimes that differ *only* in the addressing structure available (per §4). Task structure must be identical across regimes: same branching, same callbacks, same revival triggers, same closure-to-trigger distances. This makes the addressing structure the only manipulated variable.

- **Indexed regime (“code”).** The explored material carries an explicit, efficient addressing structure: stable labels on every branch and sub-result, a navigable dependency graph, search over prior steps. The index is *given*.
- **Unindexed regime (“research maths”).** The same search with no external addressing structure; any index must be built internally.

Compare $\ell^*(d, n)$, hop counts, and revival-rate across regimes.

- Flat/shallow access and intact revival in the indexed regime, but steep access and failing revival in the unindexed regime $\rightarrow$ the model leans on the *external* index and has not internalized one: self-indexing fails exactly where it would be load-bearing. This is the key indexed-vs-unindexed gap.

- Flat/sublinear access and intact revival in *both* $\rightarrow$ the model self-indexes; the external index is not what carries it.
- Steep in both $\rightarrow$ no index; an external one is merely a crutch.

Confound control. Because task structure is held fixed by construction, the remaining risk is that the addressing labels leak task information. Strip the indexed regime to *pure addressing*: opaque, content-free handles that support reference but assert nothing about branch viability, so the regime supplies navigation, not answers. Calibrate difficulty by equalizing baseline solve rates on no-callback instances before comparing the curves.

Part III: Shared methods

These apply across Stages 0–3. Stage 0 patches/probes the *given* log; Stages 1–3 patch/probe the *self-generated* trace.

10. Activation patching

Use an open-weights transformer whose activations can be cached and patched. Run the clean prompt P_c and corrupt prompt P_x , cache activations. For layer ℓ and position t , patch the corrupt run by replacing an activation with its clean counterpart, $h^{\{\ell, t\}}_x \leftarrow h^{\{\ell, t\}}_c$, and recompute the logit margin.

Patch-rescue score.

$$\rho_{\ell, t} = \frac{\Delta(P_x^{\text{patched}(\ell, t)}) - \Delta(P_x)}{\Delta(P_c) - \Delta(P_x)}$$

$\rho \approx 0$: patch did nothing. $\rho \approx 1$: patch restored the clean decision. $\rho < 0$: patch pushed further toward the corrupt decision.

Where to patch (separately): residual stream at every layer/token; attention head outputs; MLP outputs; key/value vectors at branch-status tokens; residual stream at the final query token; residual stream at explicit status/handle tokens.

Expected signatures. *Surface-text memory*: rescue only at literal status or branch-label tokens $\rightarrow$ retrieval of text, not an abstract search state. *Abstract search state*: rescue at later summary/query positions, robust under paraphrase and renaming $\rightarrow$ a compressed internal representation. *No causal state*: no patch site reliably rescues the decision.

11. Linear probing

Collect hidden states at selected positions (after each branch record, at the end of the log/trace, at the current-obligation line, immediately before the answer) and train simple probes to proof-search variables: dead-branch set D_t , frontier set F_t , current goal g_t , correct next action $a*_t$, and (Stages 1–3) the callback fact φ . Use logistic regression for $a*_t$ and g_t , multi-label logistic regression for D_t and F_t . Ground truth comes from the task generator plus an automatic parse of the trace (spot-check the parser against hand annotation).

Probe controls. Shuffled labels; random hidden states from unrelated prompts; bag-of-words baseline on the prompt text; position-only baseline; length-only baseline; train on labelled traces / test on paraphrased traces.

Interpretive ruling (load-bearing). Probing alone is not causal; it shows only that information is *present*. Because a transformer must derive whatever state it uses from its tokens, *any* sufficiently expressive decoder will recover transcript-derivable state; decodability is therefore nearly free. In particular, **a finite-state transcript parser that matches the model’s predictive accuracy does NOT falsify the hypothesis**. It shows only that the state is transcript-derivable, not that the model fails to compute and use it. The decisive evidence is **selective causal mediation**: patching one component redirects exactly the corresponding decision while non-target state, syntax, and competence stay intact. A strong result requires probe decodability

+ activation-patching rescue.

12. Attention and retrieval diagnostics

For each prompt, identify the tokens for the target branch label, the target branch status, the current obligation, and the answer-choice line. Measure attention mass from final-query tokens to branch-status tokens, $A_{\text{status}} = \sum_h \sum_{\{t \in T_{\text{status}}\}} \alpha^h_{\{q, t\}}$, and compare the clean, corrupt, and irrelevant conditions. This is diagnostic only; attention mass is not proof of causality. Use head ablation ($\Delta_{\text{ablated}} - \Delta_{\text{original}}$) to test causal involvement: a head is relevant if ablating it selectively hurts tasks requiring earlier exploration-state retrieval.

13. Latent vs. written state

Tests whether the model has a search state *before* it writes a trace, or whether the written scratchpad creates the state. Probe $\varphi, D_t, F_t, a*_t$:

1. before the model generates any trace (no-scratchpad condition, probe the final prompt-token activation);
2. after each branch exploration;
3. before the final answer.

If the variables are decodable *before* any written trace, there is evidence for latent planning. If they appear only after the trace is written, the scratchpad does the state-building; this is still consistent with the addressable-trace hypothesis (the index is then a property of the written trace, read back via attention). Stage 1's $\ell*$ curve says *how* that read-back is organized; this section says *when* the state first exists.

14. Metrics, controls, statistics

Metrics. Candidate logit margin Δ ; flip-rate; accuracy by callback distance d , trace length n , and regime; availability depth $\ell^*(d, n)$; hop count; ρ patch-rescue; handle-ablation gap; revival-rate.

Controls. All Stage-0 controls (§6.3) plus: non-handle ablation baseline (§8); difficulty-matched no-callback instances (§9); held-out *structural templates*, not just renamed symbols (§6.4).

Scaling model. Fit, for ℓ^* and for accuracy, a mixed-effects logistic regression:

$$\begin{aligned} \text{outcome} \sim & \beta_0 + \beta_1 \cdot \log d + \beta_2 \cdot \log n + \beta_3 \cdot \text{regime} \\ & + \beta_4 \cdot (\log d \times \text{regime}) + \beta_5 \cdot \text{handle_ablated} \\ & + u_{\text{template}} + u_{\text{model}} \end{aligned}$$

Index hypothesis: ℓ^* grows sub-linearly in d (β_1 small and positive, $\log d$ fits better than d); handle ablation flattens the advantage (β_5 large); regime interaction β_4 quantifies the indexed-vs-unindexed gap. Use paired bootstrap confidence intervals over instances throughout (clean/corrupt and intact/ablated are paired). For patching, report mean/median ρ , bootstrap CIs, (layer, position) heatmaps, and top-k causal sites.

Part IV: Reading the results

15. Pass / fail

Self-indexing supported if *all* hold:

1. callbacks in the model’s own trace causally flip the decision (§7.2);
2. $\ell^*(d, n)$ scales sub-linearly in d and stays bounded as n grows; $\log d$ fits materially better than linear d ;
3. routing to q is multi-hop through a small set of intermediate positions, not uniformly 1-hop;
4. ablating the model’s own handles degrades accuracy with a gap over non-handle ablation that **grows with d and n** (§8);
5. patch-rescue concentrates at handle / abstract positions, not only at literal fact tokens;
6. when a revival trigger discharges a closed branch’s reason for closure, the model reopens the *correct* branch; revival survives large closure-to-trigger distance; and path patching shows genuine re-access of the old branch span, not re-derivation (§7.4);
7. all of the above survive paraphrase, label randomisation, length matching, and held-out structural templates.

Weaker outcomes.

- 1 only $\rightarrow$ the model uses far-back facts, but flatly; no evidence of an index.
- 1–2 with constant low ℓ^* and 1-hop routing $\rightarrow$ access is direct, not hierarchical; the indexing form fails but the “addressable trace” survives in a flat form.
- handle ablation hurts but with no d/n scaling $\rightarrow$ handles help locally, not as an index.
- revival occurs only for textually-near closed branches, or only via re-derivation rather than re-access $\rightarrow$ recency / re-computation, not an index.

Falsified if callbacks do not flip decisions, or all apparent effects vanish under paraphrase / label randomisation, or φ is never decodable at q without the model re-deriving it in emitted tokens, or the model never revives a closed branch when its closure reason is discharged (then there is no internal index: “dead = forget”, and the externalized CoT *is* the only memory).

Indexed-vs-unindexed (Stage 3) is read separately: it does not pass / fail the hypothesis; it *quantifies* the indexed-vs-unindexed gap.

16. Expected outcomes

- **Outcome A: trace-only retrieval.** The model uses the trace, but patching works only at literal status tokens. Evidence for an abstract search state is weak; the model retrieves text.
- **Outcome B: abstract written-state representation.** The model uses the trace, probes decode D_t/F_t , patching later positions transfers decisions. The written trace induces a compressed proof-exploration state.
- **Outcome C: latent planning.** Search-state variables are decodable before any scratchpad is generated. The model forms some search state internally before verbalisation.
- **Outcome D: no robust state.** Behavioural effects vanish under paraphrase, label randomisation, or length matching. The apparent reasoning was a prompt artefact.

Outcome B with the §15 conditions 2–6 satisfied is the self-indexing result; Outcome B *without* the ℓ^* /handle/revival scaling is a flat addressable trace.

17. Implementation notes

- Use an open transformer with cacheable, patchable activations (a `transformer_lens`-style harness). Stages 1–3 need models strong enough to run the toy search; pre-screen with Stage 0.
- Measure with candidate log-probabilities, not sampled generation.
- Keep answer choices single letters, randomise their mapping to rules, balance which branch is correct across positions and labels.
- Keep clean and corrupt final suffixes byte-identical; for self-generated traces, clean/corrupt and intact/ablated are edits to a *frozen* trace.
- Parse the model’s own trace automatically for branch/status ground truth; spot-check against hand annotation.
- Avoid real mathematics at first; real proofs add too many confounds.

- Do not treat chain-of-thought text as faithful evidence on its own; attention maps are diagnostics, not causal proof. The decisive test is causal.
- Stage ordering: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$. Do not interpret Stage 2/3 before Stage 1's ℓ^* curve is in hand.

Stage-0 instance generator (reference).

```
from dataclasses import dataclass
import random
```

```
@dataclass
```

```
class Instance:
    rules: list[str]
    clean_log: str
    corrupt_log: str
    suffix: str
    clean_answer: str
    corrupt_answer: str
    metadata: dict
```

```
def make_instance(num_branches=2, branch_len=2, num_logged=1):
    branch_ids = random.sample(range(100, 999), num_branches)
    rule_ids = random.sample(range(100, 999), num_branches)
    viable_idx, dead_idx = random.sample(range(num_branches), 2)
```

```

rules = []
for i in range(num_branches):
    rules.append(f"R{rule_ids[i]}: S -> B{branch_ids[i]}")
    rules.append(f"R{rule_ids[i] + 1}: B{branch_ids[i]} -> T{branch_ids[i]}")

choices = ["A", "B"]
random.shuffle(choices)
viable_choice, dead_choice = choices[0], choices[1]

suffix = f""
Current obligation:
We are again at S.
Choose the next action.

Available actions:
({viable_choice}) apply R{rule_ids[viable_idx]}
({dead_choice}) apply R{rule_ids[dead_idx]}

Answer with exactly one letter: A or B.
""

clean_log = f""
Exploration log:
<BRANCH B{branch_ids[dead_idx]}>

```

```

Start action: R{rule_ids[dead_idx]}
Path: S -> B{branch_ids[dead_idx]} -> T{branch_ids[dead_idx]}
STATUS: DEAD
Reason: failed side condition C{branch_ids[dead_idx]}
</BRANCH>
"""

```

```

    corrupt_log = f"""
Exploration log:
<BRANCH B{branch_ids[viable_idx]}>
Start action: R{rule_ids[viable_idx]}
Path: S -> B{branch_ids[viable_idx]} -> T{branch_ids[viable_idx]}
STATUS: DEAD
Reason: failed side condition C{branch_ids[viable_idx]}
</BRANCH>
"""

```

```

    preamble = """
You are doing proof search in a toy calculus.
A branch is dead only if its exploration record says STATUS: DEAD.
A branch is viable only if its exploration record says STATUS: VIABLE.
When choosing the next move, avoid branches already known to be DEAD.
"""

```

```

    rule_block = "Rules:\n" + "\n".join(rules) + "\n"

```



```

return Instance(
    rules=rules,
    clean_log=preamble + rule_block + clean_log,
    corrupt_log=preamble + rule_block + corrupt_log,
    suffix=suffix,
    clean_answer=viable_choice,
    corrupt_answer=dead_choice,
    metadata={
        "viable_rule": f"R{rule_ids[viable_idx]}",
        "dead_rule": f"R{rule_ids[dead_idx]}",
        "viable_branch": f"B{branch_ids[viable_idx]}",
        "dead_branch": f"B{branch_ids[dead_idx]}",
    },
)

```

Activation-patching loop (reference).

```

clean_cache = run_with_cache(model, clean_prompt)
corrupt_cache = run_with_cache(model, corrupt_prompt)

```

```

for layer in layers:
    for pos in positions:
        patched_logits = run_with_patch(
            model, corrupt_prompt,

```

```
    patch_site=("resid_pre", layer, pos),
    replacement=clean_cache[("resid_pre", layer)][[:, pos, :],
)
delta_patched = candidate_margin(patched_logits, correct, wrong)
rho = (delta_patched - delta_corrupt) / (delta_clean - delta_corrupt)
```

18. Target claim

When a model runs a long search itself, it builds a hierarchical index over its own trace. Emitted as multi-scale handles and realized as $O(\log n)$ -depth access in its activations, this index keeps the whole search, detours and abandoned branches included, cheaply re-addressable and, when later progress warrants, revivable. This is the same capability that an external code index supplies for free.